1. Организация

2. Преговор
2.1 Упражнение 1

3. Защо двоична бройна система?

4. Компоненти на компютъра

Занятия

- Два блока седмично; във всеки от тях един учебен час теория и един учебен час практика.
- За практическите занятия Ви е необходим *компютър*. В случай, че за някой конкретен това не е така, ще предупреждавам своевременно.

Конспект - сбита версия

0. Въведение в ОС
1. Програмиране на C
2. Памет
3. Процесор
4. Файлови системи
5. Мрежи
6. Сигурност

Бройни системи

Какво е бройна система?

$$14_{(10)} = 1 \cdot 10^1 + 4 \cdot 10^0 \implies 14_{(10)}$$
$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 \implies 1110_{(2)}$$
$$14_{(10)} = 1 \cdot 3^2 + 1 \cdot 3^1 + 2 \cdot 3^0 \implies 112_{(3)}$$
$$14_{(10)} = 1 \cdot 8^1 + 6 \cdot 8^0 \implies 16_{(8)}$$
$$14_{(10)} = 14 \cdot 16^0 \implies 14_{(16)} \equiv E_{(16)}$$

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

База

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

Коефициенти

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

Позиции

Бройни системи

$$14_{(10)} = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1110_{(2)}$$

База

Коефициенти

Позиции

От значение е и базата на първоначалната бройна система база.

Бройни системи

- Най-важните бройни системи за нас са двоична и шестнайсетична.
- Първата, заради нейната тясна връзка с хардуера, върху който се изпълняват нашите програми. Ще бележим с префикс `0b`.
- Втората, защото е много лесно преобразуването ѝ към двоична и обратно. *Защо?* Ще бележим с префикс `0x`.

Бройни системи

- Ключовото наблюдение е, че една шестнайсетична цифра описва точно половин байт - т.е. четири бита. Най-голямата шестнайсетична цифра е `f`.

$$\texttt{f}_{(16)} = 1111_{(2)}$$
$$\texttt{ff}_{(16)} = 11111111_{(2)}$$

Бройни системи

- Превръщаме от двоична към шестнайсетична, разделяйки поредицата от битове (започвайки от най-младшия) на групи от по четири (наричаме ги тетрада), и на всяка от получените групи съпоставяме съответната шестнайсетична цифра.
- Обратната посока. Превръщаме от шестнайсетична към двоична, като на всяка цифра съпоставяме група от по четири бита.

Десетични представки

Представките в системата SI умножават мерната единица по степен на 10 - например $1\,\mathrm{km} = 10^3\,\mathrm{m}$, а $1\,\mathrm{ms} = 10^{-3}\,\mathrm{s}$.

Представка	**Множител**
кило (k)	10^3
мега (M)	10^6
гига (G)	10^9
тера (T)	10^{12}
пета (P)	10^{15}

Представка	**Множител**
мили (m)	10^{-3}
микро (μ)	10^{-6}
нано (n)	10^{-9}
пико (p)	10^{-12}
фемто (f)	10^{-15}

Няколко важни константи

Уговорка. За нас един килобайт е 1024 байта. Аналогично имаме

$$1\,\mathrm{KB} = 1024\,\mathrm{B}$$
$$1\,\mathrm{MB} = 1024\,\mathrm{KB}$$
$$1\,\mathrm{GB} = 1024\,\mathrm{MB}$$
$$1\,\mathrm{TB} = 1024\,\mathrm{GB}$$
$$1\,\mathrm{PB} = 1024\,\mathrm{TB}$$

Няколко важни константи

Няма да се придържаме към системата SI, която определя един килобайт като 1000 байта, а 1024 байта нарича кибибайт. Важните константи са:

$$1\,\mathrm{KB} = 2^{10}\,\mathrm{B}$$
$$1\,\mathrm{MB} = 2^{20}\,\mathrm{B}$$
$$1\,\mathrm{GB} = 2^{30}\,\mathrm{B}$$
$$1\,\mathrm{TB} = 2^{40}\,\mathrm{B}$$
$$1\,\mathrm{PB} = 2^{50}\,\mathrm{B}$$

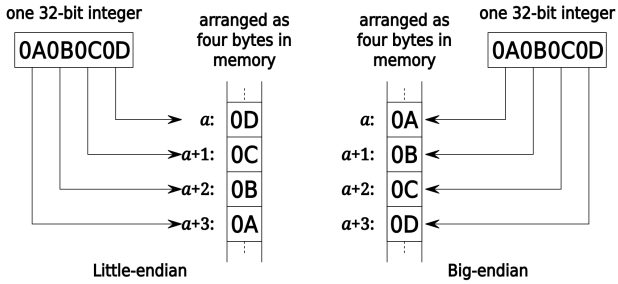
Няколко важни константи

Тяхното помнене е доста удобно, когато трябва бързо да превърнете произволно число - било то отместване във файл или големина на описвано множество от побитова карта, в размер.

Индианци и яйца (Endianness)

- *Пример.* Нека имаме целочислено число, съставено от четири байта (`uint32_t`). В паметта то се представя чрез поредица от съседни байтове - това е неговият *запис*. Програмистът влага на тези байтове *смисъл* на число, представено в двоична бройна система.
- За реда на байтовете (записа) има два общоприети варианта - отдясно-наляво или отляво-надясно; т.е. младшите битове се намират съответно или най-вдясно, или най-вляво.

Индианци и яйца (Endianness)



Фигура: Илюстрация на endianness при 4-байтово число.

Индианци и яйца (Endianness)

- Когато байтовете на данните са разположени отдясно-наляво, казваме, че се използва big endian. Обратно, ако са отляво-надясно - little endian.[1]
- При повечето архитектури (x86, ARM) се използва little-endian.
- При комуникация през мрежата се използва big-endian (още наричан network order).

[1] Терминът *endianness* произлиза от творбата „Пътешествията на Гъливер“ на Джонатан Суифт, в един епизод от която две групи лилипути имат спор за това от кой край трябва да се чупи сварено яйце - дали от малкия (little end), или от големия (big end). Оттам идват и съответно little-endian и big-endian. Действителна връзка с индианци няма. ☺

Упражнение

- Превърнете 0b111011110011 в шестнайсетична бройна система. Отг. 0xef3
- Ако горното число беше размер на файл, то колко щеше да бъде голям той? Отг. 3KB 755B
- Ако горното число беше записът в little-endian, то колко щеше да бъде размерът на файла? Отг. 0b001111111110 = 0x3fe = 1022B
- 0x8000000, 0x1000, 0x400, 0x10000. Отг. 128MB, 4KB, 1KB, 64KB
- Използвайте програма, написана на C, за да проверите дали машината Ви използва little- или big-endian за 4-байтово целочислено число. *Подсказка.* В gdb използвайте команда `x/4b &a`, където a е вашата променлива.

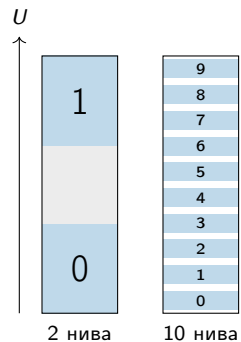
Защо двоична бройна система?

- Не винаги е било така. Машините на Бабидж (XIX в.), Harvard Mark I (1944) и ENIAC (1945) смятат в *десетична* система - за всяка цифра има колело или брояч с 10 положения.
- По същото време Z3 на Конрад Цузе (1941) и ABC на Джон Атанасов и Клифърд Бери (1942) смятат в *двоична*.[2]
- След доклада на фон Нойман за EDVAC (1945) почти всички компютри са двоични. *Защо?*

[2] Бащата на Джон Атанасов, Иван Атанасов, е българин от село Бояджик.

Защо точно две състояния?

- Сигналът е напрежение - непрекъсната величина, върху която винаги има шум.
- При 2 нива всяка стойност има широк интервал и шумът трудно ги обърква. При 10 нива интервалите са много по-тесни.
- Транзисторът не е „двоичен“ - в цифровите схеми го използваме като ключ, напълно отворен или затворен.



Двоичната логика е проста

- Две състояния са истина и лъжа - аритметиката се свежда до булева алгебра (Шанън, 1937), а И, ИЛИ и НЕ се изграждат от няколко транзистора.
- Всичко, което обработваме - числа, текст, изображения, звук, програми - може да се запише като поредица от битове.
- Не е догма. Сетун (МГУ, 1958) е троичен компютър, флаш паметите MLC/TLC/QLC пазят по 2-4 бита в клетка (4-16 нива на заряд), а PCIe 6.0 предава по 2 бита наведнъж (PAM4, 4 нива). Там допълнителните грешки се поемат от кодове за корекция.

Цели числа без знак

- С N бита записваме 2^N различни стойности. Без знак (unsigned) това са числата от 0 до $2^N - 1$.

Бита	Обхват
8	от 0 до 255
16	от 0 до 65 535
32	от 0 до 4 294 967 295
64	от 0 до $\approx 1.8 \cdot 10^{19}$

- Броят битове е фиксиран. Пренос, който не се побира, се губи - смятаме по модул 2^N. В 8 бита $255 + 1 = 0$, като километраж на кола.

Цели числа в C

Тип	Минимум по стандарт	Linux x86-64	Windows x64
`char`	8 бита	8	8
`short`	16 бита	16	16
`int`	16 бита	32	32
`long`	32 бита	64	32
`long long`	64 бита	64	64

- Стандартът гарантира само минимални размери, реалните зависят от платформата - проверете със `sizeof`.
- Когато размерът е важен, използвайте типовете от `<stdint.h>`: `int8_t`, `uint16_t`, `int32_t`, `uint64_t` и т.н.

Въпрос

```
(unsigned char)-1 == 255
```

Вярно ли е? Защо?

Отрицателни числа - първи опит

- Най-простата идея: най-старшият бит е знак, а останалите - големина (на англ. sign and magnitude).

$$+5 = 0000\,0101 \qquad -5 = 1000\,0101$$

- Два проблема:
  - две нули - $0000\,0000$ $(+0)$ и $1000\,0000$ (-0);
  - обикновеното събиране вече не работи:
    $0000\,0101 + 1000\,0101 = 1000\,1010 = -10$, а не 0.

Допълнителен код

- В допълнителен код (на англ. two's complement) най-старшият бит получава *отрицателно* тегло -2^{N-1}:

$$1111\,1011 = -128 + 64 + 32 + 16 + 8 + 2 + 1 = -5$$

- Смяна на знака: обръщаме всички битове и добавяме 1.

$$5 = 0000\,0101 \longrightarrow 1111\,1010 \longrightarrow 1111\,1011 = -5$$

- Събирането работи без промяна, преносът се изхвърля:

$$0000\,0101 + 1111\,1011 = (1)\,0000\,0000 = 0$$

Допълнителен код

- Има само една нула, а една и съща схема събира числа със и без знак.
- Обхватът е от -2^{N-1} до $2^{N-1}-1$ - за 8 бита от -128 до 127.
- Старшият бит все още показва знака (1 значи отрицателно), но не е „просто знак“ - -1 е 1111 1111, а не 1000 0001.
- Затова `(unsigned char)-1 == 255`: битовете са същите, само ги четем по различен начин.

Дроби - позиции след запетаята

- Позиционният запис продължава и надясно от запетаята - с отрицателни степени на основата:

$$0.625_{(10)} = 6 \cdot 10^{-1} + 2 \cdot 10^{-2} + 5 \cdot 10^{-3}$$

- *А в двоична?* Колко е $0.101_{(2)}$?

Дроби - позиции след запетаята

- В двоична е същото, но степените са на 2:

$$0.101_{(2)} = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.5 + 0.125 = 0.625$$

- Всяка позиция надясно е два пъти по-малка от предишната:

$$2^{-1} = 0.5 \qquad 2^{-2} = 0.25 \qquad 2^{-3} = 0.125 \qquad 2^{-4} = 0.0625$$

- Пример: $5.75 = 4 + 1 + 0.5 + 0.25 = 101.11_{(2)}$

Дроби - от десетична към двоична

- Умножаваме дробната част по 2. Цялата част на резултата е следващият бит, а с дробната продължаваме.
- *Защо?* Умножението по 2 мести двоичната запетая с една позиция надясно - първият бит след нея „излиза“ пред нея.

Пресмятане	**Бит**
$0.625 \times 2 = 1.25$	1
$0.25 \times 2 = 0.5$	0
$0.5 \times 2 = 1.0$	1

$$\Longrightarrow \; 0.625 = 0.101_{(2)}$$

- Спираме, когато дробната част стане 0.

Безкрайни двоични дроби

Пресмятане	Бит
$0.1 \times 2 = 0.2$	0
$0.2 \times 2 = 0.4$	0
$0.4 \times 2 = 0.8$	0
$0.8 \times 2 = 1.6$	1
$0.6 \times 2 = 1.2$	1
$0.2 \times 2 = 0.4$	0
...	

$$0.1 = 0.0\overline{00011}_{(2)}$$

- Повтаря се безкрайно - също като $\frac{1}{3} = 0.333\ldots$
- Краен двоичен запис има само дроб със знаменател степен на 2: $\frac{1}{2}, \frac{3}{4}, \frac{5}{8}$ - да; $\frac{1}{10}, \frac{1}{3}$ - не.
- Компютърът не може да запише 0.1 точно - пази приближение.

Къде да сложим запетаята?

- *Фиксирана запетая* - например 16 бита за цялата част и 16 за дробната. Просто е, но обхватът е малък: от ≈ 0.000015 до $\approx 65\,536$.
- Във физиката имат същия проблем и го решават с научен запис: $6.022 \cdot 10^{23}$, $1.6 \cdot 10^{-19}$. Значещите цифри и порядъкът се пазят поотделно.
- *Плаваща запетая* - научен запис, но с основа 2:

$$5.75 = 101.11_{(2)} = 1.0111_{(2)} \cdot 2^2$$

- Нормализираното число винаги започва с 1, затова тази единица не се записва (неявен бит).

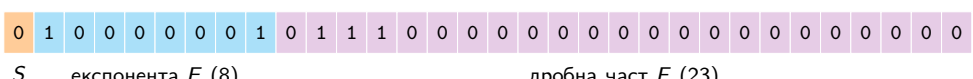
IEEE 754 - float (32 бита)

$$\text{стойност} = (-1)^S \cdot 1.F_{(2)} \cdot 2^{E-127}$$

- E се пази без знак. Отместването 127 позволява отрицателни степени: $E = 127$ означава 2^0, $E = 1$ означава 2^{-126}.
- $E = 0$ и $E = 255$ са запазени - за нулата, много малки (денормализирани) числа, $\pm\infty$ и NaN.
- `double` (64 бита): 1 + 11 + 52 бита, отместване 1023.

Пример - 5.75 като float

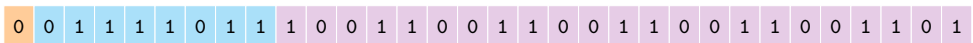
1. Двоично: $5.75 = 101.11_{(2)}$
2. Нормализираме: $1.0111_{(2)} \cdot 2^2$
3. Знак: $S = 0$, защото числото е положително.
4. Експонента: $E = 2 + 127 = 129 = 1000\,0001_{(2)}$
5. Дробна част: битовете след запетаята, 0111, допълнени с нули до 23 бита.



$$= 0\text{x}40\text{B}80000$$

А 0.1?

- $0.1 = 1.1001\,1001\,1001\ldots_{(2)} \cdot 2^{-4}$ - дробната част е безкрайна, затова след 23-тия бит закръгляме.



$$= \texttt{0x3DCCCCCD}$$

- Записаното число е 0.100000001490116119384765625 - близо до 0.1, но не 0.1.

Въпрос

```
0.1 + 0.2 == 0.3
```

Вярно ли е?

$0.1 + 0.2 != 0.3$

В който и да е език за програмиране сумата в `double` е 0.30000000000000004.

C

```
double s = 0.1 + 0.2;
printf("%d\n s == 0.3);
// 0
```

JavaScript

```
> 0.1 + 0.2 === 0.3
false
```

x86-64 (NASM)

```
a:  dq 0.1
b:  dq 0.2
c:  dq 0.3

movsd   xmm0, [a]  ; 0.1
addsd   xmm0, [b]  ; + 0.2
ucomisd xmm0, [c]  ; == 0.3?
sete    al         ; al = 0
```

Сумата е `0x3FD3333333333334`, а 0.3 е `0x3FD3333333333333` - разликата е в последния бит.

Дроби - какво следва от това

- `float` пази около 7 значещи десетични цифри, `double` - около 15-16.
- Не сравнявайте дроби с ==. Проверявайте дали разликата е достатъчно малка: `fabs(a - b) < 1e-9`.
- Не пазете пари във `float` - пазете цели стотинки в целочислен тип.
- Специални стойности: $1.0/0.0 = +\infty$, $0.0/0.0 = \text{NaN}$, и NaN не е равно дори на себе си.

Символи - ASCII

- Компютърът не знае какво е буква. Всеки символ е число, а таблица определя кое число коя буква е.
- ASCII (1963): 7 бита, 128 символа - латински букви, цифри, препинателни знаци и управляващи символи (например `'\n'` е 10).
- В C `char` е просто малко цяло число: `'A' == 65 == 0b1000001`, `'a' == 97 == 0b1100001`.
- Таблицата е подредена нарочно: малката и главната буква се различават в един бит (`'a' - 'A' == 32`), а стойността на цифра получаваме с `c - '0'`.

Символи - кодови таблици

- ASCII използва 7 от 8-те бита. Останалите 128 стойности всеки използва за своите букви - само за кирилица има CP1251, KOI8-R, ISO 8859-5 и други.
- Един и същ байт е различна буква в различните таблици:

Байт	**Таблица**	**Символ**
`0xC0`	CP1251	А
`0xC0`	KOI8-R	ю
`0xC0`	ISO 8859-5	Р

- Текст, прочетен с грешната таблица, става безсмислица („кракозябри“, на англ. mojibake).

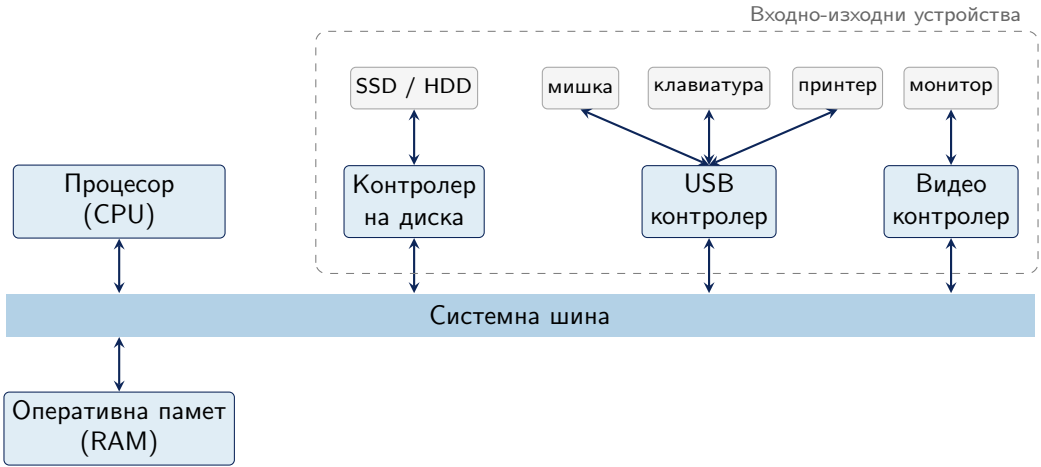
Символи - Unicode и UTF-8

- Unicode дава на всеки символ от всички писмености уникален номер (code point) от U+0000 до U+10FFFF - над 1,1 милиона възможни, от които над 150 000 вече са заети.
- В началото (1991) Unicode е 16-битов, но 65 536 символа се оказват малко и през 1996 г. е разширен.
- UTF-8 записва всеки номер с 1 до 4 байта, а ASCII символите остават същите единични байтове:

A	U+0041	`41`
А (кирилско)	U+0410	`D0 90`
емоджи „усмихнато лице“	U+1F600	`F0 9F 98 80`

- В C `strlen("Ж") == 2` - `strlen` брои байтове, не символи.

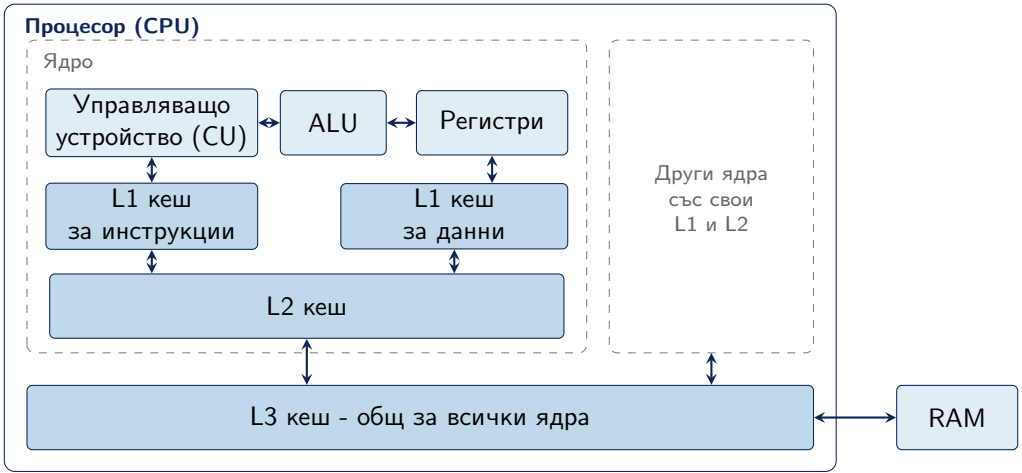
Компютърната система



Процесор и памет

- **Процесорът (CPU)** изпълнява инструкции една след друга - смята, сравнява и премества данни.
- ОС също е програма, която процесорът изпълнява. Чрез нея той управлява останалите компоненти.
- **Оперативната памет (RAM)** пази инструкциите и данните на програмите, които се изпълняват в момента - и двете в една и съща памет.
- Всеки байт в паметта има *адрес*. Такива са a, $a+1$, $a+2$ от слайда за endianness.
- RAM е енергозависима - при изключване съдържанието ѝ се губи.

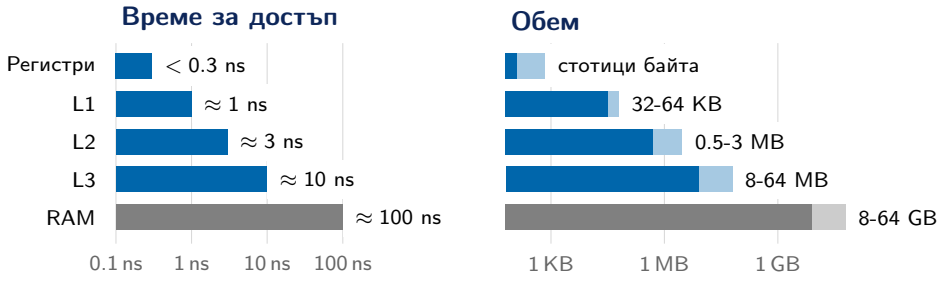
Процесорът отвътре



Части на процесора

- **Управляващото устройство (CU)** извлича следващата инструкция, декодира я и казва на останалите части какво да правят.
- **Аритметико-логическото устройство (ALU)** смята: събиране, изваждане, сравнение, AND, OR, XOR, NOT. Изградено е от логически елементи, а те - от транзистори.
- **Регистрите** са най-бързата памет - няколко десетки клетки в самия процесор, по 64 бита на x86-64.
- **Кешът** пази копие на последно използваните данни от RAM, за да не се чака паметта всеки път.

Скорост и обем по нива



Двете скали са логаритмични.

- Регистрите, L1 и L2 са във всяко ядро, L3 е общ за ядрата, а RAM е извън процесора.
- Колкото по-далеч от ALU, толкова по-голям и по-бавен. L1 е разделен на две - за инструкции (L1I) и за данни (L1D).

Регистри с общо предназначение

- Достъпни са за всяка програма - за променливи, междинни резултати и адреси.
- x86-64 има 16 такива: `rax`, `rbx`, `rcx`, `rdx`, `rsi`, `rdi`, `rsp`, `rbp` и `r8`-`r15`. ARM64 има 31: `x0`-`x30`.
- Отделно има регистри за дроби - `xmm0` от примера с $0.1 + 0.2$ е такъв.
- Някои архитектури ги делят на регистри за данни и за адреси (Motorola 68000), x86-64 и ARM64 - не.

```
mov rax, 42       ; rax = 42
add rax, rbx      ; rax = rax + rbx
```

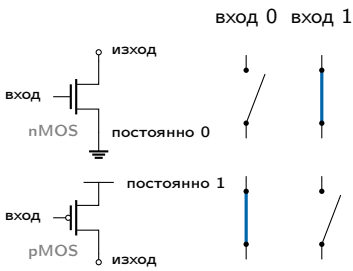
Регистри за управление

- **Програмен брояч** (PC; `rip`) - адресът на следващата инструкция.
- **Регистър на инструкцията** (IR) - текущата инструкция. Програмите не го виждат.
- **Регистър на състоянието** (PSW; `rflags` на x86-64) - резултатът от последното сравнение (нула, знак, пренос, препълване), дали прекъсванията са разрешени и режимът на процесора.[3]
- Програмата ги променя само косвено - с преходи и сравнения, а прекъсванията и режима - само ядрото.

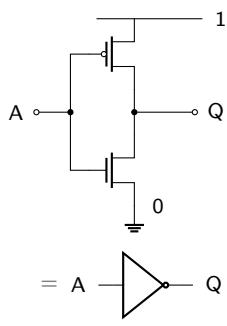
[3] На x86-64 режимът е в младшите два бита на `cs`.

От транзистора до ALU

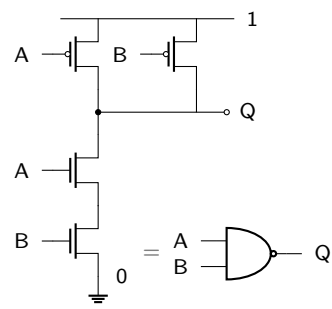
Транзисторът е ключ



НЕ (NOT)



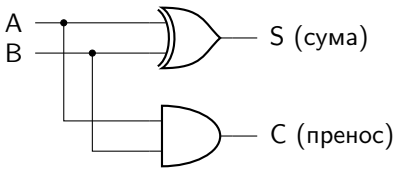
НЕ-И (NAND)



От NAND може да се построи всеки друг елемент. AND например е NAND и NOT след него - 6 транзистора.

От транзистора до ALU

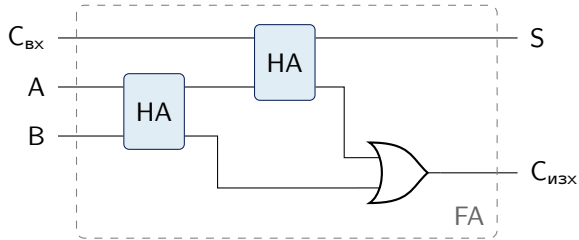
Полусуматор (HA)



A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$1 + 1 = 10_{(2)}$

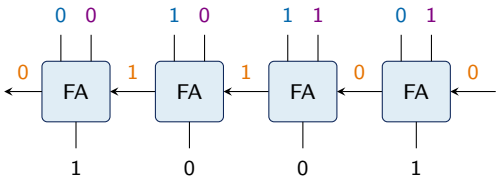
Пълен суматор (FA)



Събира два бита и преноса от предишния разред - като събиране на ръка.

От транзистора до ALU

4-битов суматор

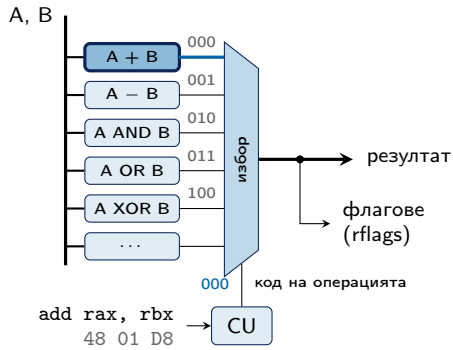


A = 0110 = 6
B = 0011 = 3
S = 1001 = 9

преносът върви отдясно наляво

Изваждане: $A - B = A + \overline{B} + 1$ - същият суматор, с обърнати битове на B и входен пренос 1.

ALU



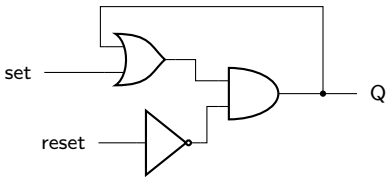
Памет и устройства за съхранение

	Пази данните без ток	Типичен обем	Време за достъп
RAM	не	8-64 GB	$\approx$ 100 ns
SSD	да	0.25-4 TB	$\approx$ 0.1 ms
HDD	да	1-30 TB	$\approx$ 10 ms

- Дисковете са енергонезависими и много по-обемни, но SSD е около 1000 пъти по-бавен от RAM, а HDD - около 100 000 пъти.
- Ако достъпът до RAM траеше 1 секунда, четене от SSD би отнело около 17 минути, а от HDD - повече от ден.

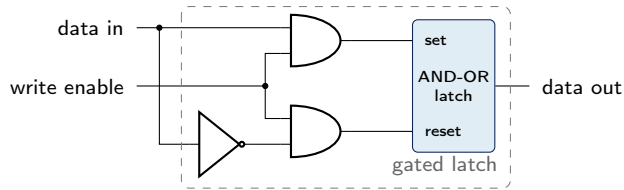
От логическия елемент до RAM

AND-OR latch



set	reset	Q	
1	0	1	записва 1
0	1	0	записва 0
0	0	Q	помни

Gated latch

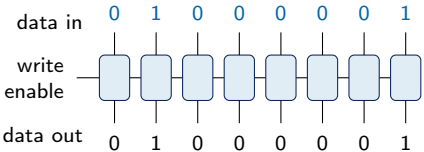


write enable = 0: изходът не се променя
write enable = 1: data out = data in

Изходът се връща обратно във входа на OR - затова схемата помни един бит, докато има ток.

От логическия елемент до RAM

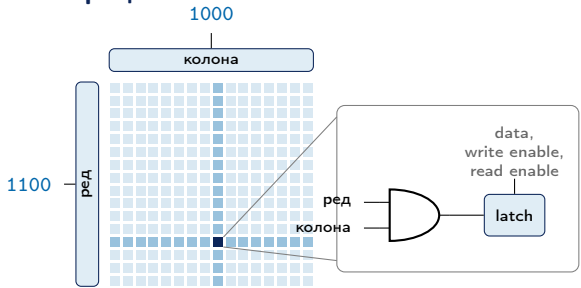
8-битов регистър



Всеки latch пази един бит,
а write enable е общ за всички.

Регистрите на x86-64 са по 64 бита.

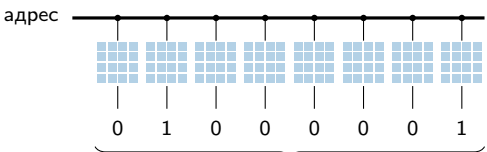
Матрица 16 × 16 = 256 бита



Адрес 1100 1000: ред 12, колона 8.
16 + 16 + 3 = 35 проводника
вместо 2 · 256 + 1 = 513

От логическия елемент до RAM

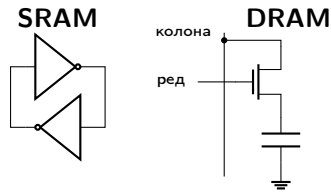
256 байта



8 матрици с общ 8-битов адрес:
256 адреса $\times$ 1 байт = 256 байта.

С 32-битов адрес - 2^{32} B = 4 GB.

Клетката в паметта



latch от 6 транзистора

бърза, но голяма

кеш

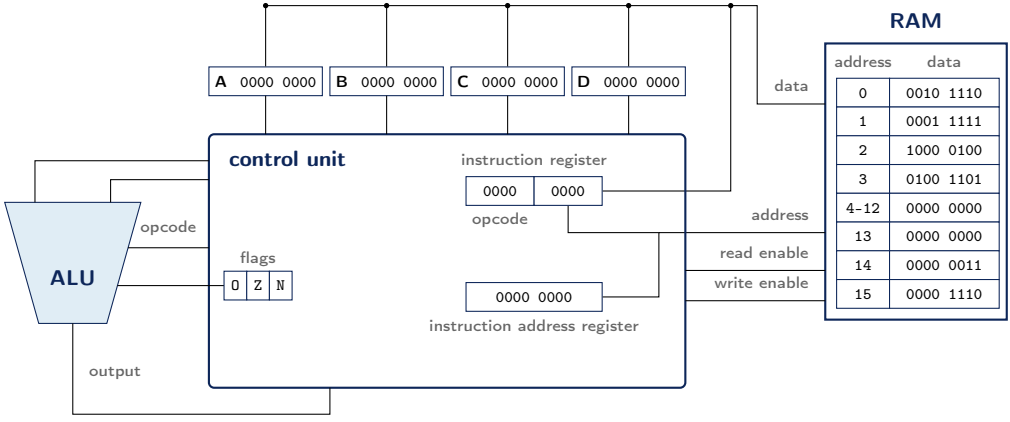
1 транзистор и 1 кондензатор

плътна, но се опреснява

RAM

Random access: всеки адрес се чете и записва еднакво бързо, в произволен ред.

От ALU и RAM до процесора



От ALU и RAM до процесора

Инструкции

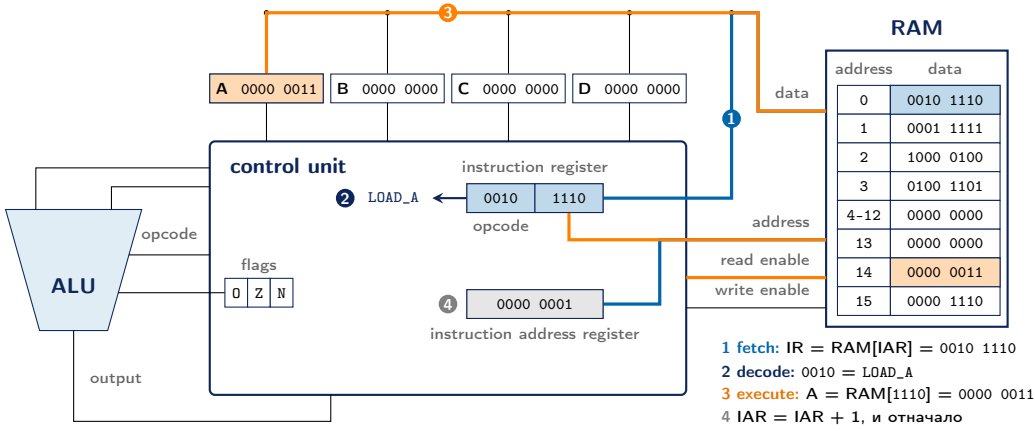
instruction	opcode	operand	действие
`LOAD_A`	0010	адрес	A = RAM[адрес]
`LOAD_B`	0001	адрес	B = RAM[адрес]
`STORE_A`	0100	адрес	RAM[адрес] = A
`ADD`	1000	регистри	втори = първи + втори

Регистри: A = 00, B = 01, C = 10, D = 11

Програмата в RAM

address	data	instruction
0	0010 1110	`LOAD_A 14`
1	0001 1111	`LOAD_B 15`
2	1000 0100	`ADD B, A`
3	0100 1101	`STORE_A 13`
...		
14	0000 0011	данни: 3
15	0000 1110	данни: 14

Изпълнение на програмата



Изпълнение на програмата

IAR	IR	instruction	A	B	RAM[13]
0000 0000		начало	0000 0000	0000 0000	0000 0000
0000 0000	0010 1110	LOAD_A 14	0000 0011	0000 0000	0000 0000
0000 0001	0001 1111	LOAD_B 15	0000 0011	0000 1110	0000 0000
0000 0010	1000 0100	ADD B, A	0001 0001	0000 1110	0000 0000
0000 0011	0100 1101	STORE_A 13	0001 0001	0000 1110	0001 0001
0000 0100	0000 0000	?	няма такава инструкция		

ADD: CU подава B и A на ALU с opcode за събиране. Изходът, $3 + 14 = 17$, отива в A, а флаговете O, Z и N са 0.

Програмата свършва на адрес 3, но процесорът не знае това - продължава с каквото има на адрес 4.

Същата програма на x86-64

Схемата		NASM	Машинен код
`LOAD_A 14`	→	`mov rax, [x]`	`48 8b 05 01 10 00 00`
`LOAD_B 15`	→	`mov rbx, [y]`	`48 8b 1d 02 10 00 00`
`ADD B, A`	→	`add rax, rbx`	`48 01 d8`
`STORE_A 13`	→	`mov [result], rax`	`48 89 05 e8 0f 00 00`

A → `rax`, B → `rbx`
адреси 13, 14, 15 → `result`, `x`, `y`

3 или 7 байта на инструкция вместо 1

```
$ ./02-toy-cpu
$ echo $?
17
```

```
mov rdi, [result]
mov rax, 60
syscall
```

Схемата спира дотук. Програмата моли ядрото да я приключи с изходен код 17.

Шина и входно-изходни устройства

- **Шината** пренася данни между компонентите: по едни линии - адреса, по други - самите данни, по трети - управляващи сигнали.
- Шината **не** е дънната платка. Дънната платка е платката, на която компонентите са монтирани и свързани.
- **Входно-изходните устройства** - клавиатура, мишка, монитор, мрежова карта, дискове - не се свързват към шината пряко, а чрез *контролер*: малък чип със собствени регистри и буфер.
- ОС говори с всеки контролер чрез *драйвер* - програмен код, който знае как да работи с точно този вид устройство.

А в действителност?

- Една обща шина за всичко има само в модела. В съвременните компютри връзките са няколко и всяка е за определен вид устройства.
- Контролерът на паметта е вграден в процесора (AMD от 2003 г., Intel от 2008 г.) и RAM е свързана към него пряко.
- Бързите устройства - видеокарта, NVMe SSD - се свързват към процесора по PCIe, всяко със собствена връзка.
- По-бавните - USB, SATA, мрежа, звук - минават през *чипсета*, който също е свързан с процесора.
- За ОС картината остава същата: процесор, памет и устройства, всяко управлявано от свой контролер.

За вкъщи

- Computerphile. *Why Use Binary?* https://youtu.be/thrx3SBEpL8. Video, YouTube
- Computerphile. *Floating Point Numbers*. https://youtu.be/PZRI1IfStY0. Video, YouTube
- Crash Course. *Boolean Logic & Logic Gates*. https://youtu.be/gI-qXk7XojA. Computer Science #3, video, YouTube
- Crash Course. *How Computers Calculate - the ALU*. https://youtu.be/1I5ZMmrOfnA. Computer Science #5, video, YouTube
- Crash Course. *Registers and RAM*. https://youtu.be/fpnE6UAfbtU. Computer Science #6, video, YouTube
- Crash Course. *The Central Processing Unit (CPU)*. https://youtu.be/FZGugFqdr60. Computer Science #7, video, YouTube

Край